

UNIVERSITÉ DE YAOUNDÉ I
Faculté des Sciences — Département d'Informatique

RAPPORT DE TP2

Tri Bitonique & Minage de Blockchain (Proof of Work)

Implémentations en OCaml, Lean 4 et OpenQASM

INF3421 — Langages Formels & Compilation

Présenté par : KAMDOM NGUETCHESSI MERVEILLE PRISCA
Matricule : 23V2060

Sous la supervision de : DR. MESSI NGUELE Thomas (PhD)

Année académique 2025–2026

I. Introduction

Ce rapport présente le travail réalisé dans le cadre du TP2, portant sur deux algorithmes en apparence très différents mais qui partagent une propriété structurelle importante : le tri bitonique et le minage de blockchain par preuve de travail (Proof of Work, PoW). Les deux sont implémentés dans trois langages aux paradigmes très distincts : OCaml (langage fonctionnel impératif, pour une implémentation exécutable et testable), Lean 4 (langage à typage dépendant, pour une formalisation rigoureuse de la spécification), et OpenQASM (langage de description de circuits quantiques, pour explorer la traduction de ces algorithmes en portes logiques réversibles).

Le fil conducteur de ce travail est la notion d'algorithme dont la structure d'exécution est indépendante des données : le tri bitonique est un réseau de tri (sorting network) — la séquence de comparaisons est fixée à l'avance, ce qui le rend directement traduisible en circuit ; le minage PoW est un problème de recherche par force brute dans un espace non structuré, exactement le type de problème que l'algorithme de Grover accélère de façon quadratique. C'est cette propriété commune qui justifie de les traiter ensemble et de les traduire tous les deux en OpenQASM.

L'objectif pédagogique poursuivi n'était pas de produire le code le plus court ou le plus optimisé, mais des implémentations aussi lisibles et progressives que possible, afin de permettre une maîtrise complète du principe algorithmique sous-jacent, condition préalable à toute traduction correcte vers un autre paradigme (fonctionnel pur, formel, ou quantique).

II. Le Tri Bitonique

II.1 Principe théorique

Une suite $(a_0, a_1, \dots, a_{n-1})$ est dite bitonique si elle croît puis décroît, ou si elle est une rotation circulaire d'une telle suite. Par exemple $(1, 3, 5, 4, 2)$ est bitonique : elle croît de 1 à 5 puis décroît de 5 à 2.

L'observation fondamentale de Ken Batchier (1968) est la suivante : si l'on divise une suite bitonique de taille n en deux moitiés, puis que l'on compare et échange chaque élément a_i de la première moitié avec l'élément $a_{i+n/2}$ de la seconde moitié (dans un sens donné), alors on obtient deux nouvelles suites, chacune bitonique, et TOUS les éléments de la première moitié sont inférieurs ou égaux à TOUS les éléments de la seconde moitié (pour un tri croissant). Ce procédé, appelé bitonic merge, peut donc être appliqué récursivement sur chaque moitié pour obtenir, in fine, une suite entièrement triée.

Pour trier une suite quelconque (non bitonique au départ), on procède par diviser-pour-régner : on trie récursivement la première moitié en ordre croissant et la seconde moitié en ordre décroissant ; l'assemblage des deux forme alors une suite bitonique, à laquelle on applique le bitonic merge. C'est exactement la structure implémentée dans les trois langages de ce rapport.

La complexité du tri bitonique est de $O(n \log^2 n)$ comparaisons, ce qui est moins bon qu'un tri par comparaison optimal en séquentiel ($O(n \log n)$, atteint par exemple par le tri fusion). Son intérêt n'est donc pas la performance séquentielle, mais le fait que la suite des comparaisons est fixe et connue à l'avance : elle ne dépend pas des données. C'est cette propriété qui permet de la représenter comme un

circuit (câblage figé de comparateurs) et qui la rend hautement parallélisable — d'où son usage historique sur les premiers ordinateurs parallèles et, aujourd'hui, sur GPU.

II.2 Implémentation OCaml

Le fichier `ocaml/bitonic_sort.ml` contient trois fonctions centrales, dans l'ordre de complexité croissante :

`compare_and_swap`

C'est la brique de base du réseau de tri : elle compare deux éléments du tableau, aux positions i et j , et les échange si leur ordre ne correspond pas au sens (croissant ou décroissant) voulu.

```
let compare_and_swap (a : int array) (i : int) (j : int) (dir : sens) : unit = let doit_echanger = match dir with | Asc -> a.(i) > a.(j) | Desc -> a.(i) < a.(j) in if doit_echanger then begin let tmp = a.(i) in a.(i) <- a.(j); a.(j) <- tmp end
```

`bitonic_merge`

Étant donné un segment du tableau déjà bitonique ($a.(bas..bas+n-1)$), cette fonction le transforme en une suite triée. Elle compare chaque élément de la première moitié du segment avec l'élément situé à distance $n/2$ dans la seconde moitié, puis s'appelle récursivement sur chacune des deux moitiés obtenues.

`bitonic_sort_rec`

Cette fonction implémente le diviser-pour-régner décrit en II.1 : elle trie récursivement la première moitié en croissant, la seconde en décroissant, puis appelle `bitonic_merge` sur l'ensemble.

Le programme de test principal trie un tableau de 8 éléments puis un tableau de 16 éléments en ordre décroissant, afin de vérifier le fonctionnement dans les deux sens. L'exécution donne :

```
Tableau initial : 5 3 8 1 9 2 7 4 Tableau trié : 1 2 3 4 5 7 8 9 Trié (décroissant, n=16) : 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
```

Une précondition importante, vérifiée par `is_power_of_two` avant l'appel, est que la taille du tableau doit être une puissance de 2 : c'est une contrainte structurelle du découpage en moitiés égales utilisé par l'algorithme classique.

II.3 Formalisation Lean 4

Le fichier `lean/BitonicSort.lean` reprend exactement la même structure algorithmique (`compareEtEchange`, `bitonicMerge`, `bitonicSort`), mais dans un style fonctionnel pur : au lieu d'un tableau muté en place, on manipule des listes immuables (`List Nat`), et `compareEtEchange` renvoie une nouvelle paire au lieu d'échanger en mémoire.

Deux propriétés de spécification sont énoncées, qui définissent formellement ce que signifie « trier correctement » :

- `trierBitonique_trie` : le résultat produit est effectivement une liste triée (au sens du prédicat `estTrie`) ;
- `trierBitonique_permutation` : le résultat est une permutation exacte de l'entrée — aucun élément n'est perdu, dupliqué ou inventé.

Ces deux théorèmes sont énoncés avec le mot-clé `sorry`, c'est-à-dire acceptés sans démonstration complète. Ce choix est délibéré et assumé : une preuve complète en Lean/Mathlib de la correction du tri bitonique nécessite une induction combinée sur la profondeur de récursion et sur la structure de la suite bitonique produite à chaque étape, ce qui représente un développement de plusieurs centaines de lignes dépassant largement le cadre d'un TP2. L'objectif pédagogique atteint ici est la

formulation précise et correcte de la spécification formelle — étape indispensable avant toute preuve — plutôt que la preuve elle-même.

II.4 Traduction en circuit OpenQASM

Le fichier `openqasm/bitonic_sort.qasm` illustre comment le réseau de comparateurs du tri bitonique se traduit en portes quantiques réversibles, sur un exemple simplifié à 4 éléments d'un seul bit chacun (valeurs dans $\{0,1\}$, portées par des qubits en état de base, sans exploiter de superposition).

La traduction de `compare_and_swap` repose sur le principe suivant : un échange conditionnel (SWAP contrôlé, ou porte de Fredkin) entre deux qubits, dont la condition de contrôle — « $a[i] = 1$ et $a[j] = 0$ » pour un tri croissant — est calculée au préalable sur un qubit ancilla à l'aide d'une porte de Toffoli (CCX). Après le swap conditionnel, l'ancilla est « défait » (uncompute) pour rendre le circuit réutilisable et strictement réversible.

Le circuit implémente les trois étapes du réseau de tri bitonique pour $n=4$: l'étape 1 forme une suite bitonique (comparaison croissante sur la paire (0,1), décroissante sur la paire (2,3)) ; les étapes 2 et 3 appliquent le bitonic merge à distance 2 puis à distance 1, exactement dans le même ordre que l'arbre d'appels `bitonic_merge/bitonic_sort_rec` côté OCaml.

Une limite est explicitement documentée dans le fichier : la version d'uncompute présentée y est simplifiée à des fins pédagogiques ; une version rigoureusement réversible nécessiterait de dupliquer la valeur d'un qubit avant le swap (via un CNOT vers un qubit propre) pour pouvoir uncomputer l'ancilla avec la valeur antérieure au swap, plutôt qu'avec la valeur postérieure. Cette nuance est un point d'attention classique en conception de circuits réversibles.

III. Le Minage de Blockchain (Proof of Work)

III.1 Principe théorique

Une blockchain est une liste chaînée de blocs, où chaque bloc contient des données, le hash du bloc précédent, et un nonce (nombre arbitraire modifiable). Le hash d'un bloc est calculé à partir de la concaténation de tous ses champs ; toute fonction de hachage cryptographique a la propriété d'effet avalanche : changer un seul bit de l'entrée change environ la moitié des bits de sortie, de façon imprévisible.

Miner un bloc consiste à trouver un nonce tel que le hash obtenu satisfasse une contrainte de difficulté — typiquement, commencer par un certain nombre de zéros. Comme le hash est imprévisible, il n'existe pas de raccourci connu : la seule stratégie est d'essayer des nonces successifs jusqu'à trouver une solution valide. C'est cette preuve de travail (Proof of Work) qui rend coûteuse la falsification d'un bloc déjà miné : modifier ses données change son hash, ce qui invalide le chaînage avec le bloc suivant, sauf à re-miner tous les blocs suivants — un coût de calcul dissuasif à l'échelle d'un réseau.

Le nombre moyen d'essais nécessaires pour trouver un nonce valide croît exponentiellement avec la difficulté : pour une contrainte de d zéros hexadécimaux, il faut en moyenne 16^d essais, puisque chaque caractère hexadécimal du hash a 1 chance sur 16 d'être un zéro, indépendamment des autres (propriété d'uniformité d'une bonne fonction de hachage).

III.2 Implémentation OCaml

Le fichier `ocaml/blockchain_mining.ml` utilise le module `Digest` de la bibliothèque standard OCaml (MD5) à la place de SHA-256, pour rester sans dépendance externe — le principe algorithmique du minage est rigoureusement identique quelle que soit la fonction de hachage utilisée.

`calculer_hash`, `hash_valide`, `miner`

`calculer_hash` concatène les champs du bloc (index, données, hash précédent, nonce) et en calcule l'empreinte MD5 hexadécimale. `hash_valide` vérifie que cette empreinte commence par le nombre de zéros requis par la difficulté. `miner` incrémente le nonce en boucle jusqu'à obtenir un hash valide, et renvoie le nombre d'essais effectués — une mesure directe du coût de calcul du Proof of Work.

`chaîne_valide`

Cette fonction parcourt la chaîne de blocs et vérifie deux propriétés indépendantes : l'intégrité (le hash stocké de chaque bloc correspond bien au hash recalculé à partir de son contenu actuel) et le chaînage (le champ `hash_precedent` de chaque bloc correspond exactement au hash du bloc qui le précède).

Le programme de test mine une chaîne de trois blocs avec une difficulté de 4 zéros hexadécimaux, puis simule une fraude en modifiant les données du bloc 1 sans le re-miner. Résultat obtenu :

```
Bloc 0 (genese) mine en 83692 essais -> hash = 000063424819acff2f4ae66721df140c Bloc 1 mine en 68847 essais -> hash = 0000a13e82756122760a44118f04625d Bloc 2 mine en 19328 essais -> hash = 0000ce6ee2e4b98595a866ad85625c4d
Chaîne valide ? true Apres modification frauduleuse du bloc 1, chaîne valide ? false
```

Ce résultat illustre concrètement la propriété fondamentale d'une blockchain : toute modification a posteriori d'un bloc, même minime, est détectable, car elle rend son hash stocké incohérent avec son contenu réel.

III.3 Formalisation Lean 4

Le fichier `lean/BlockchainMining.lean` modélise la structure `Bloc` (miroir exact du type `bloc` OCaml), ainsi que les prédicats `chaînee` (chaque bloc pointe correctement vers le hash de son prédécesseur) et `integre` (chaque bloc est valide vis-à-vis d'une fonction de hachage abstraite `h`). La fonction de hachage réelle (MD5) n'est pas ré-implémentée en Lean : elle est modélisée par un paramètre abstrait, ce qui est suffisant pour raisonner sur les propriétés structurelles de la chaîne, indépendamment du choix précis de l'algorithme de hachage.

Le théorème central, `fraude_detectee`, énonce que sous l'hypothèse que la fonction de hachage est sans collision (injective), toute modification des données d'un bloc déjà miné, sans re-minage, rend la chaîne invalide. Ce théorème est volontairement énoncé sous hypothèse explicite d'injectivité : la résistance aux collisions d'une fonction de hachage cryptographique réelle est une hypothèse de sécurité computationnelle (aucune collision n'a été trouvée en un temps raisonnable), et non un fait démontrable mathématiquement — la distinction entre les deux est elle-même un point important de la théorie de la cryptographie, explicitement mise en évidence par cette formalisation.

III.4 Illustration quantique (Grover)

Le minage PoW est structurellement un problème de recherche non structurée : trouver, parmi N nonces possibles, celui (ou ceux) qui satisfait une contrainte, sans autre information que la possibilité de tester chaque candidat individuellement. C'est exactement le problème que résout l'algorithme de Grover, avec une accélération quadratique : $O(\sqrt{N})$ évaluations de l'oracle au lieu de $O(N)$ en classique.

Le fichier `openqasm/grover_mining_nonce.qasm` illustre ce principe sur un exemple minimal à 2 qubits ($N=4$ nonces possibles), où un unique nonce $n^*=11$ est marqué par un oracle de phase (construit à partir d'une porte de Toffoli et de portes de Hadamard sur un qubit ancilla), suivi d'un unique diffuseur de Grover — une seule itération suffisant à amplifier fortement la probabilité de mesurer n^* , pour $N=4$.

Une limite importante est discutée : ce circuit ne recalcule pas un vrai hash SHA-256 (dont la traduction en portes quantiques réversibles représenterait plusieurs milliers de portes, hors de portée pédagogique ici) ; il illustre uniquement le principe algorithmique. Le rapport souligne également que l'accélération quantique réelle du minage Bitcoin reste hors de portée des ordinateurs quantiques actuels (ère NISQ : nombre de qubits et taux d'erreur insuffisants pour exécuter un oracle SHA-256 complet), ce qui est un point de discussion académique reconnu dans la littérature sur la sécurité post-quantique des blockchains.

IV. Résultats et discussion

Les implémentations OCaml sont exécutables et testées : les deux fichiers compilent et produisent les résultats attendus, présentés en II.2 et III.2. Le tri bitonique est validé sur deux tailles différentes (8 et 16 éléments) et dans les deux sens (croissant, décroissant). Le minage est validé par la détection réussie d'une fraude simulée sur la chaîne.

Les formalisations Lean 4 sont syntaxiquement correctes et reproduisent fidèlement, dans un style fonctionnel pur, la structure des algorithmes OCaml. Les propriétés de correction sont énoncées avec précision ; leur démonstration complète est laissée pour un travail ultérieur, le choix pédagogique de ce TP ayant été de privilégier la maîtrise de la spécification formelle plutôt que l'automatisation de sa preuve.

Les circuits OpenQASM démontrent la traductibilité de principe des deux algorithmes vers un modèle de calcul quantique, tout en documentant honnêtement leurs limites : simplification à de petites tailles, uncompute pédagogique simplifié pour le tri, et absence de véritable oracle SHA-256 pour le minage. Ces limites sont volontairement explicitées plutôt que masquées, conformément à l'exigence de rigueur scientifique.

V. Conclusion

Ce TP a permis d'implémenter et de comparer, sur deux algorithmes structurellement apparentés (réseau de tri fixe et recherche non structurée), trois approches complémentaires : une implémentation exécutable et testable (OCaml), une formalisation de la spécification dans un système à typage dépendant (Lean 4), et une traduction en circuit quantique (OpenQASM) qui met en évidence pourquoi ces deux algorithmes précisément — et pas n'importe quel algorithme — se prêtent à une telle traduction : indépendance de la structure de calcul vis-à-vis des données pour le tri bitonique, accélération quadratique de Grover pour la recherche de nonce.

Au-delà de l'exercice technique, ce travail illustre une idée plus générale de la théorie des langages et de la compilation : un même algorithme peut être exprimé dans des paradigmes très différents (impératif, fonctionnel pur, formel, quantique), et c'est précisément la compréhension profonde de sa structure — indépendante de toute syntaxe particulière — qui rend ces traductions possibles et correctes.

Annexe — Organisation des fichiers

Le code source complet, commenté en français et organisé pédagogiquement, est disponible dans les fichiers suivants, livrés avec ce rapport :

- `ocaml/bitonic_sort.ml` — tri bitonique, exécutable, testé
- `ocaml/blockchain_mining.ml` — minage PoW, exécutable, testé
- `lean/BitonicSort.lean` — spécification formelle du tri
- `lean/BlockchainMining.lean` — spécification formelle de la blockchain
- `openqasm/bitonic_sort.qasm` — réseau de tri en portes quantiques ($n=4$)
- `openqasm/grover_mining_nonce.qasm` — recherche de nonce par Grover (jouet, $n=4$)
- `README.md` — instructions de compilation et d'exécution pour les trois langages